H. F. Oliveira Rocha, *Practical Event-Driven Microservices Architecture*
https://doi.org/10.1007/978-1-4842-7468-2_6

6. Dealing with Concurrency and Out-of-Order Messages

Hugo Filipe Oliveira Rocha[1]
(1) Ermesinde, Portugal

This chapter covers:

- Why tackling concurrency in a monolithic application is different than tackling it in a distributed microservice architecture
- The impacts of concurrency in distributed event-driven services
- The differences between pessimistic and optimistic concurrency and when to use each
- How to apply pessimistic and optimistic concurrency strategies
- Using event versioning to handle out-of-order messages
- Applying end-to-end partitioning to avoid concurrency and out-of-order messages
- How to avoid hotspotting as a consequence of routing events

In Greek mythology, when Zeus sent Pandora to earth to be married, he wanted to punish the human race for accepting fire from Prometheus. Pandora, oblivious of his real intent, gratefully accepted his wedding gift, a mysterious jar that Zeus warned never to open. Pandora, who was created to be curious, couldn't resist the desire to open the jar. When she finally did open it, all the world's greatest evils and life's most devious afflictions flew from that jar and cursed the world: envy, pain, hatred, war, event consumption concurrency, and unordered messages. However, Pandora was able to close the jar before the last thing flew out, the thing that killed message partitioning.

Mythology apart, concurrency is often a dreaded topic, an obscure concern banished to a scarce number of use cases and to those unlucky enough to need to face it. Reasoning with code that runs in parallel is

challenging and often requires a trained mind. Concurrency issues can occur when two threads or services use the same resource or change the same state simultaneously (typically referred to as race conditions). Race conditions are often hard to understand, replicate, and tackle consistently.

Usually, race conditions are very challenging to pin down due to happening very sparingly. I'm sure you came across the argument "well, this will probably never happen" a time or another. There are probably race conditions on the software used today due to never being detected or the user just ignoring it and restarting the app or refreshing the page. However, when you move to a high-throughput, high-availability ecosystem with hundreds of changes per minute, that "probably never" becomes "almost certain." For example, I had a team struggling with a bug that occurred once every one million calls; the service consumed approximately one hundred messages per second, which means it happened on average nine times per day. Now I can't even tell you how hard it was to reproduce locally. If I had to design an experience to prove Murphy's law, event-driven systems would likely be a viable candidate. The "everything that can go wrong will go wrong" plays an essential role in these situations. Event-driven services usually handle enormous loads turning near impossible chances into very likely, which many of the issues are related to parallel message processing. These issues must be dealt with upfront and must be tackled in the solution design with deliberate strategies. This chapter works through those strategies and explains how you can apply them to event-driven services.

We often use and work with sequential programs and applications without the need to worry about concurrency. In truth, without the performance requirements and relevant scale, concurrency is mostly unnecessary. Concurrency is performance; we use concurrent execution to increase throughput or reduce execution times, often to use all available physical resources and turn the system as fast as it can be. Performance often only becomes a concern with the relevant scale. Some applications might avoid concurrency issues; however, in event-driven architectures, they are an omnipresent concern.

Performance aside, concurrency can also become a concern with parallel requests to the same resource, for example, two users buying the same product at the same time. Monolithic applications traditionally tackle this issue with the traditional ways to deal with concurrency, like locking resources. We also often deal with this at the database level, for example, with transactions.

Messages are consumed in parallel and, most likely, by several instances of the same service. Each service needs to maintain a valid state while processing multiple events that access the same resources simultaneously. Processing multiple messages that access the same entities across different instances raises a different problem than we usually face in monolithic applications.

Concurrency is often a primary concern in event-driven services due to the scale and the performance requirements. The way we handle concurrency needs to have the lowest performance impact it possibly can. Event-driven services are also horizontally scalable; the services that can extensively use the available physical resources and provide satisfying performance have better scaling properties. Having both acceptable performance and linear scaling is essential in managing eventual consistency and increasing demand.

As we discussed at the beginning of this chapter, Pandora closed the box right before the last thing came out. In the original mythology, it was the thing that killed hope. We distorted the story and said the last thing to fly from the box was the thing that killed message partitioning, which, as you will see in Section 6.6, is the equivalent of hope in event-driven architectures.

6.1 Why Is Concurrency Different in a Monolith from an Event-Driven Architecture?

There are many ways to deal with concurrency in traditional single-process applications. However, it's a substantially different challenge dealing with concurrency in a distributed architecture. It is essential to understand this difference when dealing with event-driven services; not

being aware of the parallel event processing consequences can trigger some difficult corner cases and intricate bugs. This section will detail the differences between managing concurrency in a single-process application, like a monolith, and managing it in an event-driven service.

As discussed in Chapter 1, monoliths are single-process applications where all applications' functionality is deployed together. Often, all the application requests go through the same single application. In this type of application, we can use the standard strategies to deal with concurrency. Let's illustrate this with an example.

Let's say we are buying a product on an eCommerce platform . When we submit a new order, the application has to save the order information and check the available stock for the item we just requested. The product can have stock in several different warehouses. For example, if we were buying a shirt with size M, we could have a US warehouse with one stock unit, another one in the UK with one stock unit, and another one in Japan with five stock units. To minimize the order's shipping costs, the application would have to retrieve the available stock from every warehouse and apply a routing algorithm to understand which warehouse to retrieve the stock. Once it figured out which warehouse to use, it would decrease the stock and ship the order. Figure 6-1 illustrates this example.

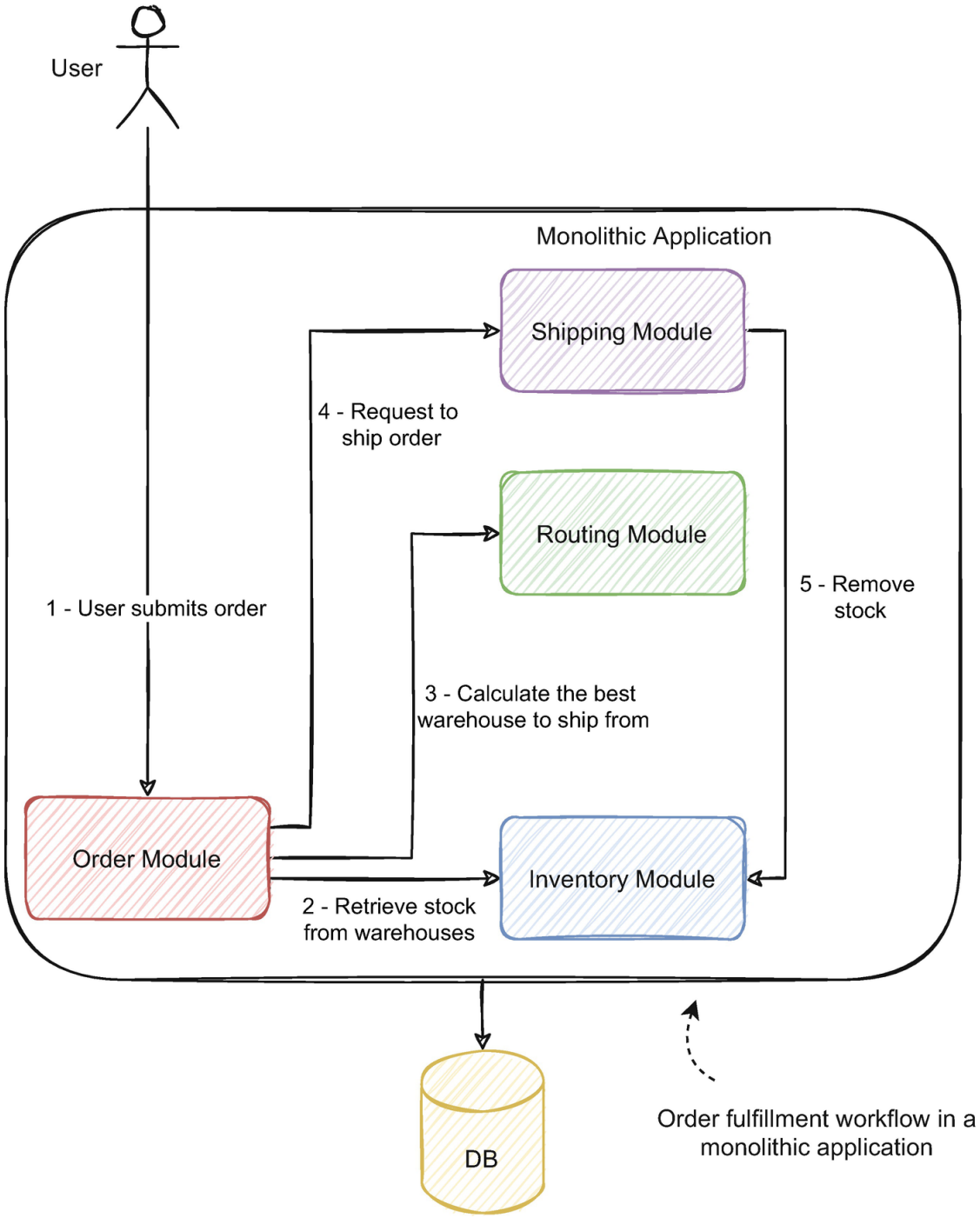

Figure 6-1 Example of an order fulfillment flow in a monolith

The order fulfillment process follows the flow we just discussed:

1. The user submits an order which the order module receives.
2. The order module requests the stock information from every warehouse from the inventory module and validates if there is enough stock to fulfill the order.
3. The order module requests the routing module to calculate the best warehouse for that customer's address and the available warehouses.
4. When the routing module calculates the best warehouse, the order

module requests the shipping module to ship the order.
5. The shipping module (could also be the order module if we were following the orchestrator pattern) requests the stock changes to the inventory module and ships the order.

Hopefully, our eCommerce platform doesn't have one user at a time using it; that would mean it had only minimal success, although it would make things a lot easier. A random number of users would be browsing and submitting their own orders. Let's say another user submitted an order for the same item we were buying simultaneously. The application would receive and process both orders in parallel. Unfortunately, both orders are from the United States, and the product we are buying has only one stock unit left in the closest warehouse. With the preceding process in place, the system would fulfill which order?

If we are unlucky enough, and we are always unlucky enough in production, it could be both orders. Imagine if two orders go through the workflow in parallel, the order module would retrieve the stock and request the routing module to calculate the best warehouse to retrieve the stock. Let's say there is one stock unit available in the US warehouse and one stock available in the UK warehouse. If the users placed both orders in the United States, the routing module would likely answer that the best warehouse for both orders was the warehouse in the United States. When advancing, both orders would retrieve stock from the same warehouse. Since only one stock unit is available, one order would either advance for an unexisting stock unit or would fail, depending on the implementation. Despite the fact, there might still be stock available to fulfill that order in other warehouses.

In a single-process application, it is easier to manage this issue, and there are several strategies we can use. We can use a pessimistic approach and lock the order processing for each product (or product size, depending if the products vary by size). If we receive a second order for a product we are already processing, it will have to wait for the processing of the first order to finish. An important detail is to lock per product; we can still process different products in parallel; we just avoid processing

the same product simultaneously. This detail ensures the application retains some of the performance and isn't severely impacted.

Alternatively, we could use an optimistic approach and assume there is no concurrency. The workflow would normally run without any lock or validation. In the last step, the shipping module could detect if no stock was available and request the routing algorithm to run again with the latest stock, retriggering the process.

We could also partition our requests so that the application never processes orders for the same product simultaneously. This way, it is possible to build highly concurrent, highly scalable systems without the need for locks or retries; we solve concurrency by architecture rather than implementation. We won't go into much detail on how to achieve this now, but we will work through a practical example at the end of the chapter and detail this concept.

How is concurrency different in event-driven services? In fact, some of these strategies are still applicable in event-driven services, at least in some use cases. Others can't be applied due to the distributed and horizontally scalable nature of the services. Let's work through an example to illustrate this. Figure 6-2 shows the order created event consumption workflow in the inventory service.

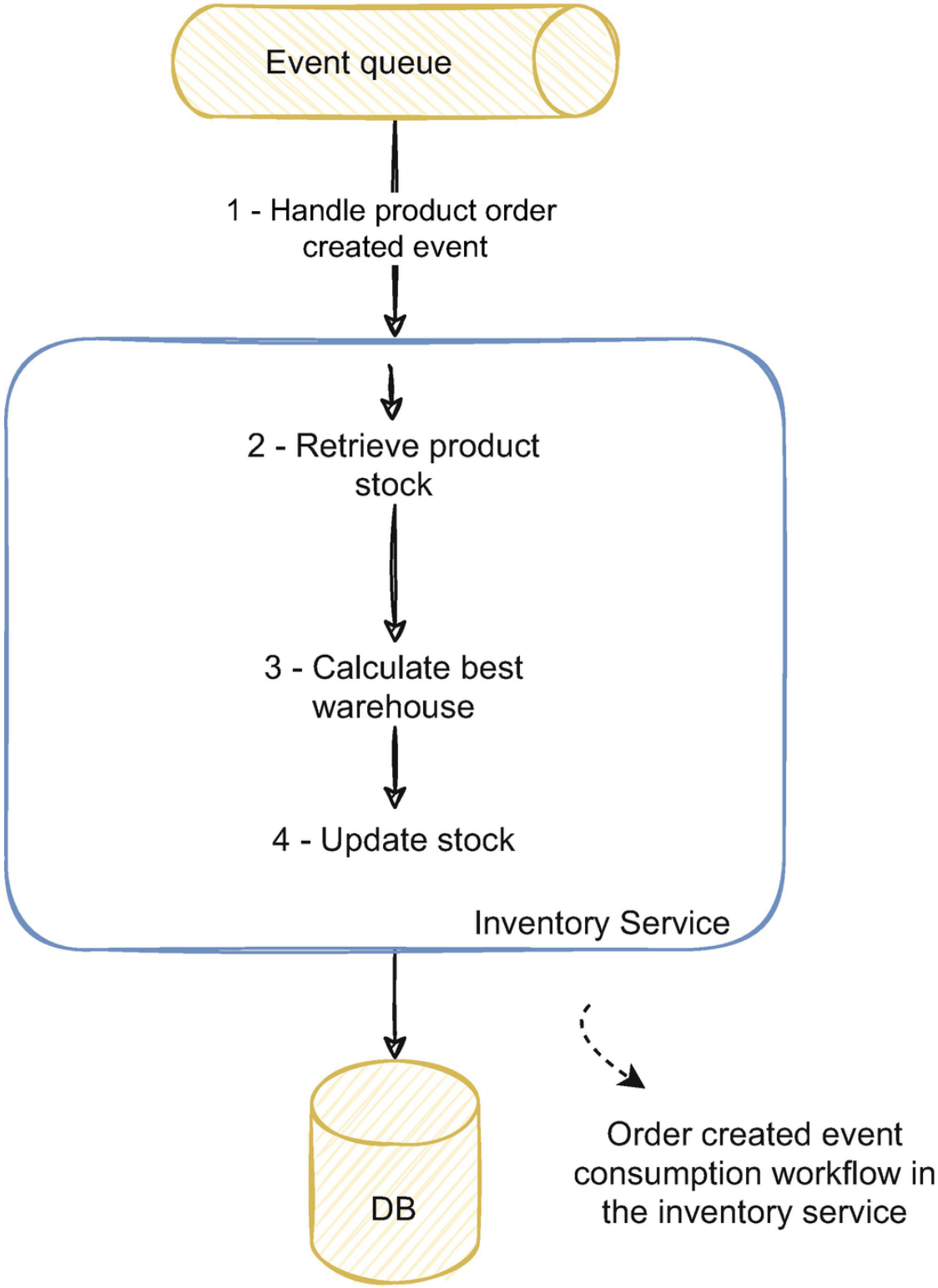

Figure 6-2 Consumption of the order created event for a given product inside the inventory service

The inventory service needs to change the stock for each order submitted to the system. To do that, it handles order created events from an event queue. For each event, the service has to retrieve the current product's available stock from all warehouses, calculate the best warehouse to satisfy the order, and update the product's stock in that warehouse.

In this example, we can have a concurrency issue the same way we did in the last example with the monolithic application. For example, if we receive two events for orders created for the same product simultaneously, the service will fetch the product's stock for both orders and calculate the best warehouse. If the warehouse is the same, the service will try to retrieve stock from the same warehouse simultaneously, potentially originating a concurrency issue if there isn't enough stock.

For some services, it might be sufficient to work in a single thread and completely avoid concurrency issues; however, this isn't the most frequent use case. As we discussed at the beginning of the chapter, we design event-driven services to be horizontally scalable and respond promptly to varying load or usage needs. We often adopt event-driven architectures to have systems that have these characteristics. The need to scale and deal with increasing amounts of requests usually means we need performant services. Single-threaded services won't likely cut it; as we discussed, concurrency is performance.

We can horizontally scale single-threaded services, but we would likely need a much higher number of services than we would need when using concurrency and parallelism. Even when hosted in cloud providers, single-threaded services won't probably use all the available physical resources. Depending on what the services do, they often only use a modest percentage of the CPU and memory, even on lower-tier machines. As you might guess, this isn't cost-effective, and we aren't making the best usage of the resources we have available. Also, by being single-threaded, we can't tune the service to make the most of the available resources. When using parallelism, we can adjust the number of services, and the degree of parallelism in each service to optimize the minimum number of machines (or containers) needed with the max usage of physical resources, optimizing the cost and performance metrics accordingly.

The example pictured in Figure 6-2, and the concurrency issue it can produce, has the same nature as the one we discussed when using a single-process application, illustrated in Figure 6-1. In what way an event-driven service is different from the situation we discussed before? Having multiple instances of the same service can add an additional layer of complexity when tackling concurrency. Figure 6-3 illustrates this situation.

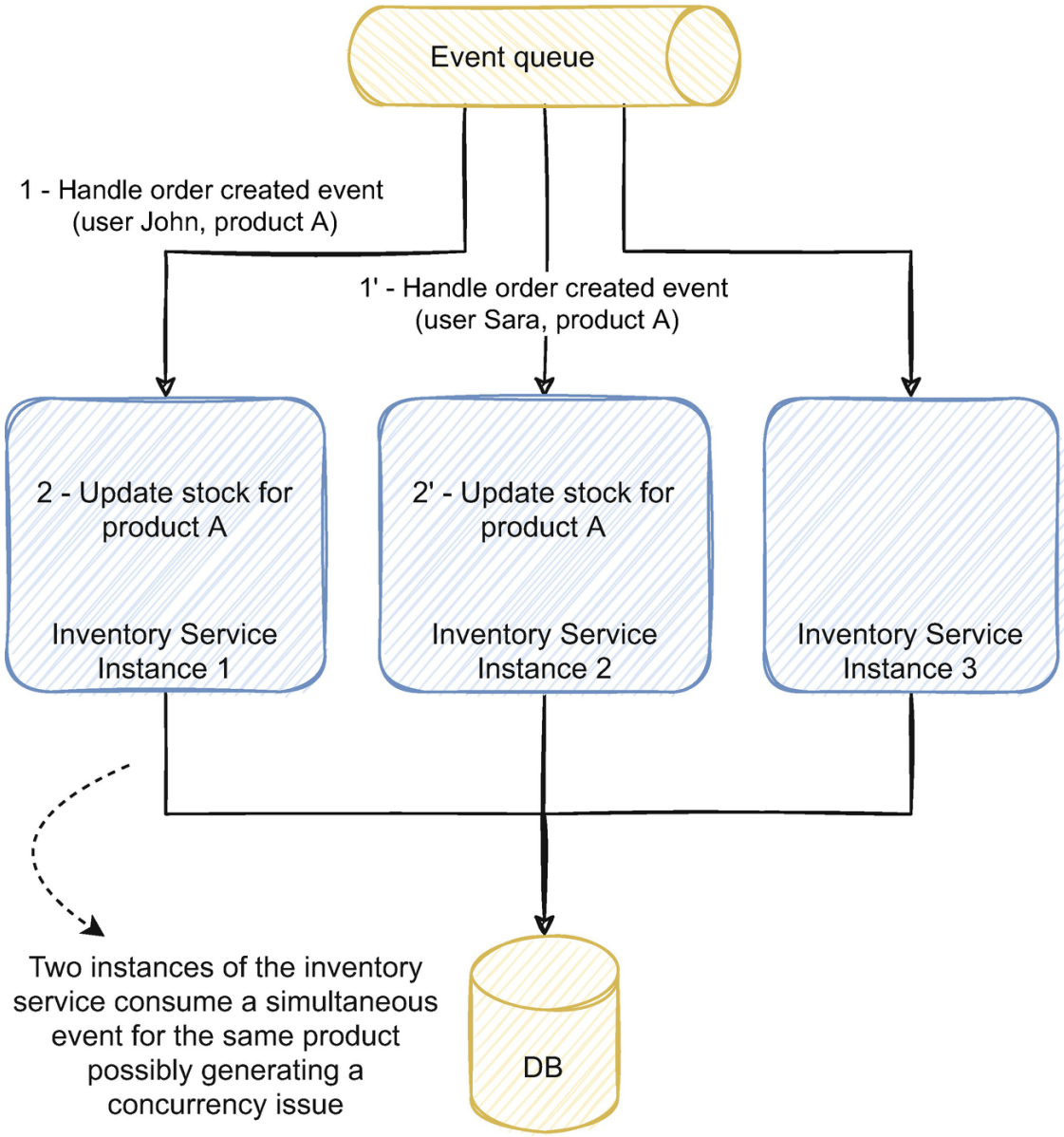

Figure 6-3 Multiple instances of the inventory service consuming events in parallel

Figure 6-3 illustrates three instances of the inventory service . All three instances process events independently from each other. They can also be processing multiple events in parallel inside each instance, so we have two parallelism degrees, inside each instance and between instances. In this example, the service receives two order created events simultaneously for the same product (product A), one for user John and one for user Sara.

Inside a service's instance, we can use the same strategies we would use in a single-process application like a monolith. But they don't guarantee we won't have concurrency because other instances might be simultaneously processing concurrent changes. For example, if we locked product A in instance 1, the lock would only work on that instance ; instance 2

would have no knowledge of it due to being a separate independent process.

To an inexperienced developer, this kind of concurrency problem might seem far-fetched and unlikely. To be fair, some of these problems have a minimal chance of happening. How likely is it to have two different users buying the same variant of a product with the limited stock at the exact same time? Often it comes down to windows of a few milliseconds. Is it worth it to solve a problem with such low chances of happening, even if it happens occasionally? There are solutions to problems that are more costly than living with the bug happening once or twice per year.

In the end, it's all about scale. We discussed in Chapter 1 how event-driven architectures could be the answer for challenging scalability problems and might be best applied when an existing application struggles with a given scale. When we are consuming dozens, hundreds, or thousands of events per second, it's not about how it likely won't happen. It will, several times per hour or minute.

It's not just about saying our solution is performant and horizontally scalable, it's also about saying that it is consistent and trustworthy at scale. No matter how much scale the business will need in the future, this kind of issue won't happen no matter how much load you throw at it. We can't take concurrency lightly, and we need deliberate and sustainable strategies to tackle it from the ground up.

6.2 Pessimistic vs. Optimistic Concurrency, When and When Not to Use

Traditionally, there are two types of strategies to deal with concurrency, pessimistic and optimistic. In fact, I propose a third, which is avoiding concurrency altogether, handling concurrency by design as opposed to handling concurrency by implementation. This section will detail what differentiates pessimistic and optimistic concurrency, when to use each, and how the architecture and data model design can avoid concurrency altogether.

Pessimistic and optimistic concurrency is the difference between apologizing and asking permission .[1] Pessimistic strategies request access to the resource and will only act based on the response, while optimistic strategies assume they have access to the resource, and when they don't, they apologize by acting accordingly.

Let's illustrate this with the example we discussed in the last section. In Figure 6-2, we discussed how the inventory service had to fetch the stock, calculate the best warehouse, and update the stock for that warehouse. We could take a pessimistic approach by locking the product's stock quantity during the operation. Or we could assume there is no concurrency, calculate the best warehouse, and try to update the stock. If the stock changed in the meantime, we could fail or retry the operation.

Pessimistic concurrency strategies lock the resource upfront. The services request access to the resource; if it is available, the application locks it, and it becomes unavailable for every other access that might happen until the application frees the resource. Suppose the resource is unavailable (another consumer or thread already has a lock on it). In that case, the access will fail, and the application will either wait to release the lock or fail the operation.

Optimistic concurrency strategies assume there is no concurrency and act when concurrency occurs. Typically, there is no locking involved; the flow of the application runs without synchronization. When the application is about to persist the changes, it validates if anything has changed since the start of the operation. If it did, the application aborts or retries the operation.

6.2.1 Pessimistic vs. Optimistic Approaches

A pessimistic approach is the easiest to reason with since once the application acquires the lock, we can reason with the program's flow in a single-threaded mindset. This attribute dramatically eases the way we understand and develop the service's logic. However, it might have a considerable impact on performance. Since we are locking a resource and preventing other requests from accessing it, the higher the number of re-

quests for the same resource, the higher the time it will take the application to respond.

An optimistic approach typically doesn't involve any kind of locking and doesn't block the application. The lack of locks, under certain circumstances, can lead to much higher performance; under the right conditions (we will detail them next), it will behave as no concurrency prevention strategy is in place. However, every time a write collision is detected, it has to retry the operation, which can be costly. Also, reasoning with concurrent code while developing the service's logic is more complex and can easily introduce bugs without a constant concern on concurrency issues. Retries must also be idempotent, a common concern in event-driven services, which we will detail in Chapter 7.

Retrying the operation is often more expensive than acquiring and maintaining a lock. For example, in the situation we discussed earlier, using a pessimistic approach and locking the inventory service's execution will affect performance. But retrying the operation when the service detects conflicts implies the service has to fetch the stock, calculate the best warehouse, and try to save the information again. If this happens frequently enough, it will likely be slower than restricting the resource's access through locking. Imagine if we have ten concurrent users trying to buy the same product, in an optimistic approach, one of the ten would succeed, but the other nine would fail and would have to retry. On the next retry, some would likely fail and would have to retry again. In these situations, it would be best to lock the resource upfront and mediate the access.

Optimistic strategies are only cost-effective if the chances of the operation succeeding are sufficiently high. A good rule to follow is to use a pessimistic approach when there is a high chance of conflicting requests and use an optimistic approach when the chances are low. If it is very likely that a request conflicts, we restrict access to the resource, which guarantees orderly access and prevents consecutive retries. In use cases where it is still possible but unlikely for the requests to conflict, we can use an optimistic approach and enjoy the same performance as if we have no mechanism to prevent concurrency to begin with, unless for the unlikely

requests that cause concurrency. Using a pessimistic approach in an environment with low chances of concurrency may severely impact the performance for a small percentage of occurrences; thus, the cost-benefit ratio is smaller.

6.2.2 Solving Concurrency by Implementation and by Design

For example, suppose our eCommerce platform has a small quantity of highly sought-after products with thousands of active users, and the chances of several users ordering the same product simultaneously are high. In that case, we might benefit more from using a pessimistic approach. On the other hand, suppose products are expected to be on sale for a more extended period, like a real estate selling platform, where houses are on sale for considerable periods with a handful of users actually requesting to buy the house. In that case, an optimistic approach might be best.

As with most solutions , it isn't a black and white kind of choice. We can use both approaches or combine them as we see fit. There might be use cases in the architecture, or even in a specific service, that benefit from applying a pessimistic approach in one place and an optimistic one in another.

Often, the hard part is not about deciding which kind of approach is more beneficial but rather how long it stands to the varying needs through time. Different systems and applications exhibit different patterns of usage. The likelihood of concurrency and the conflict patterns often change over time; a highly concurrent use case might be so during a limited window of time. For example, our real estate platform might have low concurrency patterns, unless for sporadic, once-in-a-lifetime deal houses that have a vast demand and attract hundreds of users at the same time. It's complex to have a strategy that varies not by use case but by specific timespans.

To deal with these cases, concurrency is best solved by design rather than implementation. In the specific case of event-driven services, routing messages and using end-to-end partitioning is the best approach as it

avoids concurrency as a whole in a performant and transparent way. However, as we discussed before, event-driven architectures often aren't composed of only event-driven services. Sometimes, in the real world, we need to combine both asynchronous and synchronous functionalities. In those use cases, we might need to use the traditional optimistic or pessimistic strategies, but they are the exception rather than the rule.

Solving concurrency by design isn't always possible or practical. As we will detail in Section 6.6, it also has several limitations that might not apply to every use case. Handling concurrency by design relies on being able to design the system in a way concurrency is impossible. In event-driven architectures, it is often based on event routing. Routing events isn't always possible in various use cases, for example, when we don't have a suitable routing key or integrate from external systems. In those situations, solving concurrency by implementation is preferable. Concurrency by implementation is also simpler to reason with since it's similar to the traditional concurrency handling approaches. In the following sections, we will approach by implementation strategies first since they will exercise your thought process toward concurrency issues, are easier to grasp, and are valuable tools for you to have in your toolbox when by architecture strategies are impractical.

Data modeling also plays a part in concurrency; the way we model the data can impact the performance and how we deal with concurrency. In Chapter 3, we discussed aggregates' size and how changes to the same aggregate shouldn't be done in parallel when we detailed DDD. The aggregate size also impacts how concurrent a system is, thus affecting the system's performance. For example, let's say our platform sells clothes, and we can choose to design the aggregate to reflect either a product or a product's size (like an XS). If we choose the product granularity, we can process different products simultaneously, and if we choose the size granularity, we can process different sizes, even from the same product, simultaneously. The smaller aggregate size has higher throughput but also has all the implications in the consumers that we discussed in Chapter 5 and will further detail in Chapter 8.

Finally, the strategies to handle concurrency we discuss in this chapter are in the scope of several distributed instances of the same service (e.g., several inventory service instances). You might ask how to solve concurrency across different services (e.g., between the order and inventory service). These strategies (except end-to-end partitioning, as you will see) aren't effective across different services. Although we can use them, one service shouldn't lock another; they are self-sufficient and should be able to scale independently. To manage concurrency across different services, we should use a higher-level approach, like a Saga, as discussed in Chapter 4.

6.3 Using Optimistic Concurrency

Section 6.2 discussed how optimistic and pessimistic concurrency prevention strategies work and when to apply them. But how do we use them in practice? In this section, you will learn how to apply optimistic concurrency as we work through a practical example.

Let's look at the same example we discussed in Section 6.2 involving the inventory service. When processing an order created event, the service has to retrieve the product stock, calculate the best warehouse, and remove the stock for that product in the corresponding warehouse. Figure 6-4 illustrates this process when the service processes two order events for the same product.

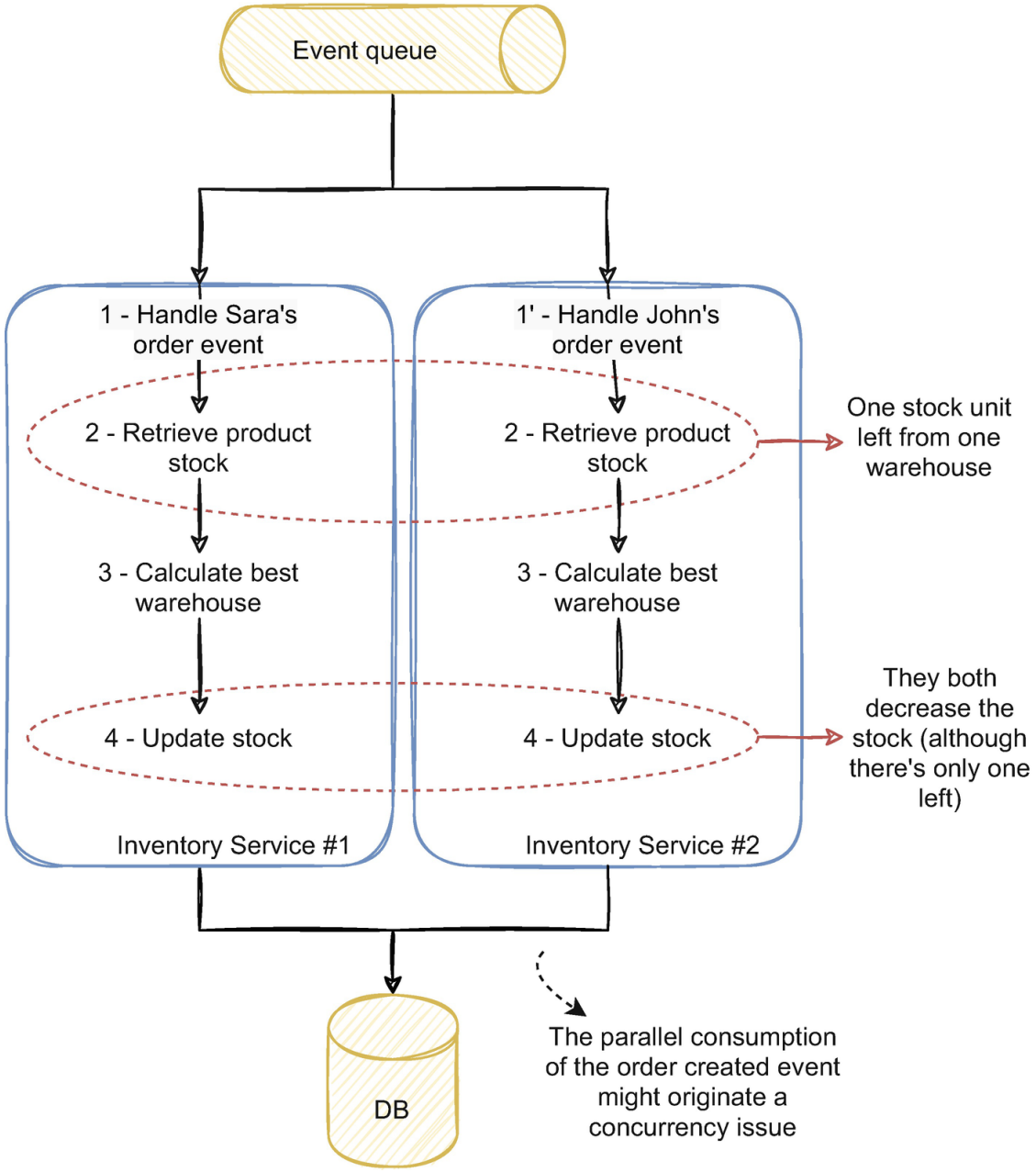

Figure 6-4 The inventory service consuming two order created events, from two different users, for the same product

Two instances of the service process simultaneously one order created event each: one from Sara and one from John. Both ordered the same product, which has only one stock unit left in one warehouse. If the fourth step that updates the stock only decreases the quantity, the product will end up with negative stock. An immediate solution could be to add an additional step before updating the stock to check whether there was still enough stock to perform the operation. Nonetheless, we would only decrease the chances of having concurrency issues and wouldn't remove it entirely. As we discussed, even if the chances are small, the scale and number of events the service processes will make an unlikely inci-

dent certain. Even if we currently have low scale, we leave the chances for it to happen in the future when the scale and throughput increase.

How could we solve it using an optimistic approach? As we discussed in Section 6.2, optimistic prevention strategies assume there is no concurrency and act when concurrency is detected. In this case, when the service is updating the stock, it would only apply the change if the stock for that product wasn't changed since we fetched it in step 2. If it was, the service wouldn't decrease the stock and would, for example, retry the operation.

Let's work through this solution if the service was using SQL Server , for example. The inventory database would save the stock information in a table containing information about the product, warehouse, and stock quantity. Listing 6-1 illustrates an example schema of this table.

```
1    -- Create table stock
2    CREATE TABLE Stock #A
3    (
4        ProductId int,
5        WarehouseId int,
6        Quantity int,
7        LatestChangeTimestamp bigint, #B
8        primary key (ProductId, WarehouseId)
9    )
10
11   UPDATE Stock SET Quantity = Quantity - ? WHERE ProductId
= ? AND WarehouseId = ? #C
12   UPDATE Stock SET Quantity = ? WHERE ProductId = ? AND
WarehouseId = ? #D
#A Stock table definition
#B Unix timestamp column
#C Update stock by decreasing the current quantity from the
ordered amount
#D Update stock by setting the quantity to the calculated
value by the service
```

Listing 6-1 Stock table schema and possible updates

After fetching the stock and calculating the warehouse, the inventory service would update the quantity column in this table. It could do it in two ways, by choosing one of the two updates in Listing 6-1. If the service updated the stock using the first, the stock quantity could turn negative

(1-1 and then 0-1). If it used the last, both operations would succeed by setting the stock to zero, although there was only one quantity available.

With an optimistic approach, we need to know if the data changed when doing the second update. We could do that using the LatestChangeTimestamp column. Every time the stock quantity changes, that column is also updated with the timestamp of the change. In step 2 of Figure 6-4, the service fetches the current stock along with the timestamp of the last update. In step 4, the service only changes the stock if the timestamp is still the same. It could use an update operation similar to the one illustrated in Listing 6-2.

```
1     UPDATE Stock SET Quantity = ?
3     WHERE ProductId = ?
4     AND WarehouseId = ?
5     AND LatestChangeTimestamp = ? #A
#A Version clause
```

Listing 6-2 Optimistic update operation

With the condition using the LatestChangeTimestamp, the update will only apply if the timestamp is still the same; otherwise, it won't find and consequently won't affect any row. We used the timestamp, but it could be an incremental version or another field that made sense. The Unix timestamp is a straightforward property to use as a version since we easily generate it from a timestamp; it's an integer and always increases. After the operation, if no rows were affected, then the record was the target of concurrency, and we could retry the operation. In this case, the last order wouldn't be fulfilled due to lack of stock.

Optimistic approaches like we detailed here are very simple and straightforward to use. They don't rely on a specific technology or external dependency. If we understand the mechanism, we can apply it regardless of the technology. When concurrency is an issue, this can be a simple and performant solution.

In this example, we used SQL Server , but it can be applied in most database technologies. In fact, this strategy is used under the hood by many technologies. For example, ElasticSearch has a way to manage concurrency using a sequence number[2] (the same version or timestamp we discussed) to address concurrency in a very similar way we detailed here. Cassandra has the concept of lightweight transactions[3] that is very similar to the mechanism we just discussed. The concept of optimistic locking[4] is

also discussed in the context of Cassandra, which uses the same similar approach. NEventStore,[5] a popular event sourcing framework, which can be used with the persistence engine for MongoDB, uses the same concept to manage concurrent changes to the same aggregate. EntityFramework also has an optimistic concurrency management approach[6] similar to what we discussed.

This approach's simplicity and flexibility makes it a viable solution to have in our toolbox and apply it where we see fit. As we discussed in Section 6.2, optimistic concurrency shines in environments with low chances of concurrency. Environments with operations that are likely to conflict and how they benefit from pessimistic approaches are detailed next.

6.4 Using Pessimistic Concurrency

Pessimistic concurrency prevention strategies are usually the standard way to deal with concurrency. They are easier to reason with since they guarantee no other operation happens simultaneously. This section will discuss how to use a pessimistic approach to handle concurrency. We will work through a practical example similar to the one we discussed when detailing optimistic concurrency in Section 6.3.

Let's use the same example illustrated in Figure 6-4. Two instances of the inventory service fetch, manipulate, and update the stock quantity for the same product. In a single-process application (e.g., if we had only one instance of the inventory service), the typical approach would be locking a resource while the service is processing that call. But since an in-memory lock would only work for the local instance, it isn't the right approach when we have two or more service instances.

6.4.1 Distributed Locks in Event-Driven Microservices

A similar approach we could use is a distributed lock. A distributed lock works the same way a local lock would (like a mutex) but relies on an external dependency to manage the lock across several instances. Figure 6-5 illustrates this approach in the same example we discussed before.

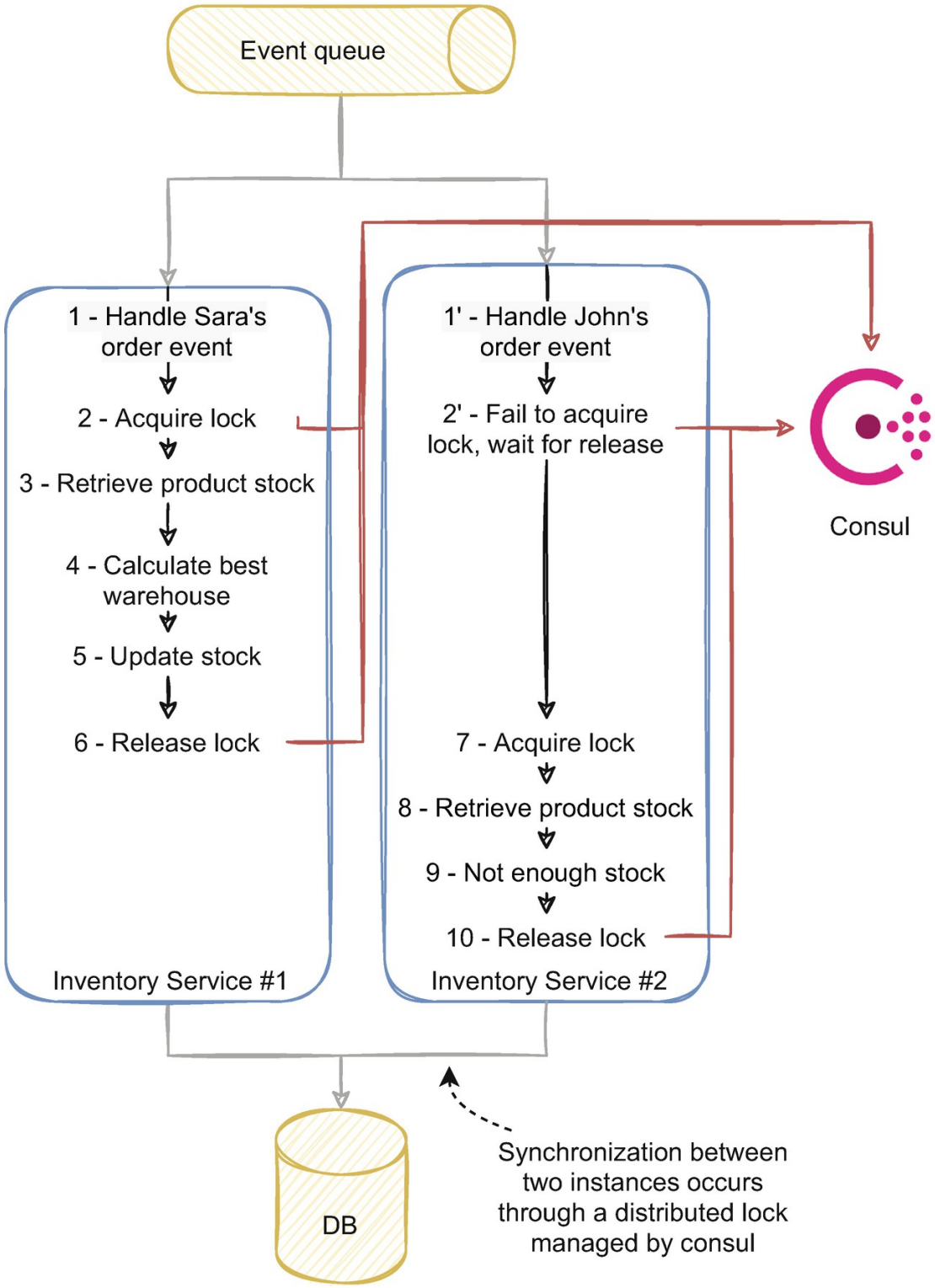

Figure 6-5 A distributed lock applied to the same example with the inventory service

The first instance handling Sara's event acquires the lock successfully. Then it proceeds to process the event by fetching the stock. The second instance receives John's order and tries to obtain the same lock. Since the first instance already acquired it, the service will block until it is released. Meanwhile, in the first instance, Sara's order is fully processed and removes the only stock unit left from the product. The first instance advances to the last step and releases the lock. By releasing the lock, the sec-

ond instance starts to process John's order and fetches the stock. Since Sara's order already removed the stock, the second instance will reject the order.

Let's deep dive into how this situation could work in the code. Listing 6-3 illustrates how we could implement the lock logic in the inventory service.

```
1    public async Task Consume(
2            CancellationToken cancellationToken,
3            OrderCreatedEvent orderCreatedEvent)
4        {
5             // Generate lock key based on product id
6             var lockKey =
orderCreatedEvent.ProductId.ToString(); #A
7
8             // Create consult client and lock
10            var distributedLock =
consulClient.CreateLock(lockKey);
11
12            try
13            {
14                // Acquire lock for that product
15                await
distributedLock.Acquire(cancellationToken); #B
16
17                // Fetch product's stock
18                var stockList = await this.stockRepository
19                    .GetStockAsync(orderCreatedEvent.ProductId);
20
21                // Calculate best warehouse for that order
22                var bestWarehouseId = this.routingService
23                    .CalculateBestWarehouse(stockList,
orderCreatedEvent.OrderId);
24
25                // Change stock for that warehouse
26                await this.stockRepository
27                    .UpdateStockAsync(
28                        orderCreatedEvent.ProductId,
29                        bestWarehouseId,
30                        orderCreatedEvent.Quantity);
31            }
32            catch (Exception)
33            {
```

```
34                 // Any exception occurring during the execution
of the process
35                 // is handled here
36             }
37
38             // Release the lock
39             await distributedLock.Release(cancellationToken); #C
40         }
#A Generate lock key
#B Acquire lock based on the generated key
#C Release the lock once the operation finishes
```

Listing 6-3 Acquiring and releasing lock in inventory service

The way we create and release the lock is very similar to how it would work in a local, in-memory lock using a mutex or a lock statement. Instead of managing the lock locally, the service relies on an external dependency to manage the synchronization, in this case, Consul. The statements using the distributedLock are requests to Consul. In the distributedLock.Acquire method, the instance requests to acquire the lock; if it succeeds, it will advance with the remaining operations. If the lock is already in use by another instance, it will block until it is freed. The instance releases the lock and makes it available for other instances once the service completes the event processing, with the distributedLock.Release method.

An important point is how we generate the lock key. In this example, the lock key is the product id, illustrated by the lockKey variable. Instead of generating a different one on each event, we could choose a static key, for example, "InventoryServiceKey." That would mean the service would try to lock the event consumption independent of the event's contents for every event. This way, the service would act as a single thread service, and the service's performance would deteriorate significantly. We also wouldn't gain much from blocking the consumption for every event since concurrency issues might occur for processing simultaneous changes for the same product, not across different products.

Using the lock key as the product id guarantees only one instance is processing one product simultaneously, but different products are processed concurrently. It is an important detail since it dramatically benefits the service's performance. This detail also relates to what we dis-

cussed in Chapters 3 and 4 about the aggregate size. If the aggregate scope is considerably large, it will negatively affect the service's performance since it limits the ability to have higher levels of concurrent changes.

We used Consul to manage the lock, but there are other options. ZooKeeper, a popular tool to provide distributed synchronization and configurations, is another viable option. Kafka in the past used ZooKeeper to manage information about the cluster, topics, and partitions across nodes. It also can provide fully distributed locks that are globally synchronous.[7] Redis can also be an option, and there are several implementations of distributed locks with Redis.[8] Martin Kleppmann also has an interesting article[9] about fencing locks approach with Redis, which is worth checking if you want more details about the implementation and its impacts.

You might recall when we discussed the Jepsen tests in the previous chapters and their usefulness in validating distributed systems' safety. In fact, the Jepsen tests found issues with Consul in the past,[10] for example, and although they are surpassed and Consul fully complies[11] with the Jepsen tests, they raise an interesting concern on how our distributed services might interact with an external lock management system. In fact, what would happen if the service crashes, is unavailable, or suffers a network partition? In that case, Consul would free the lock, but that instance of the service might still change information concurrently with other instances.

Kyle Kingsbury has a fascinating talk[12] about distributed databases that highlight many of these problems and the associated limitations of distributed locks. Distributed locks might be useful in some use cases and, as we discussed, can be a straightforward way to apply a pessimistic approach to concurrency. However, they also suffer from problematic limitations and can lead the service to process invalid states. Techniques to handle inconsistency and transient errors can help, like event idempotency and versioning, which will be detailed in Chapter 7. They also introduce an additional dependency of a third-party tool to manage locks. Distributed locks can be useful when used sparingly and in specific use cases

where locking is a must. However, in most use cases, other alternatives are more straightforward and less impactful.

6.4.2 Database Transactions As a Concurrency Approach in Distributed Microservices

Perhaps the most common way and what first comes to mind as a solution to deal with concurrency issues is the use of transactions. In fact, they can be a very straightforward and performant way to solve some concurrency issues. When we horizontally scale a service and add more instances, they all share the same database. We can use it as a point of synchronization between all instances, and transactions are easy to reason with to most developers due to their widespread usage. We are excluding here the distributed transactions and the two-phase commit protocol we discussed in Chapter 4.

How could we solve the concurrency issues we detailed with the inventory service in Figure 6-5? We could open a transaction when fetching the stock, calculate the best warehouse, and commit the transaction when updating it. This way, the database guarantees no other instance changes the same information while the transaction is running. Often we hear the issues of transactions and the performance impacts they have; however, more often than not, it's more a theoretical concern than a practical one. Transactions are a very optimized tool that can achieve fantastic performance even in high-throughput systems. Relational databases are often associated with monolithic databases (as we discussed in Chapters 4 and 5), but they are as valid as any other database when applied in a specific service if that service benefits from using it.

Some databases also enable locking and unlocking the service based on a given resource specified by the application. One example of this is the sp_getapplock[13] from SQL Server, which can act as a distributed lock much like what we discussed with Consul. We observed some success with some use cases using this approach. Still, there are more sustainable and straightforward ways to deal with concurrency than using the database as a distributed lock.

Although useful and straightforward, transactions can be troublesome with long-running operations and severely affect the system's performance. The example we discussed was relatively simple, but imagine if calculating the best warehouse involved more complex operations or communication with external systems. The transaction might lock data for an unfeasible amount of time.

Also, the transaction management in this kind of system tends to leak toward the application or domain logic. In practice, how could we replace the lock for a transaction? One of the easiest ways is to wrap the operations in a TransactionScope (in C#). If we exchanged database technologies, would that code live unchanged as it was supposed to? Likely it wouldn't. To be fair, we don't change database technologies often, if ever. However, it hints at how easily the database's transactional logic spreads to the domain and application logic, which can make the code difficult to maintain.

Transactions are also limited to the technologies that support them. Many NoSQL databases don't support transactions. Overall, if we are using a database that uses transactions, it can be a simple way to deal with concurrency issues and a quick fix to some pressing issues. Still, as we saw when discussing distributed locks, a better solution is to avoid concurrency altogether, as we will also discuss at the end of the chapter.

6.5 Dealing with Out-of-Order Events

A related concept that can also derive from concurrency is out-of-order events. Until now, when we discussed the service consumption from an event stream, we always assumed the events in the stream were in order. However, this isn't always the case; services can consume unordered events due to several reasons. This section will discuss how events can become unordered and how we can deal with and mitigate their impacts.

Why does event ordering matter? Let's illustrate with an example. Let's say we have a product service that has an aggregated view of the product's information and exposes that information to other services. It also handles stock events from the inventory service and saves the prod-

uct and stock information locally in a denormalized read model. The inventory service publishes stock changed events every time a product's stock changes. For example, a given product has three stock units, and the users buy it two times. The inventory service will publish one stock changed event with remaining stock quantity two for the first purchase and a stock changed event with remaining stock quantity one for the second one. The product service handles those events and updates the product stock to quantity two and then to quantity one similar to what we discussed in Figure 6-5.

But what if the two events arrive in an incorrect order? If the earliest event (the stock changed to quantity two) arrives after the latest (the stock changed with quantity one), the product service will update the stock to quantity two, which doesn't reflect the product's real quantity.

6.5.1 How Can Events Lose Their Order?

Incorrect ordering can happen due to several reasons. Multiple publishers for the same queue can also have a desynchronized clock. Clock synchronization across machines is typically done with the NTP (network time protocol); the considerations about this protocol and how it achieves clock synchronization across machines are beyond the scope of this book; however, the median for most providers is in the millisecond range or under. For most use cases, this synchronization is enough and provides satisfactory results.

Besides clock synchronization issues, the event stream might not be in the correct order in the first place. Ordering issues can also be related to the message broker technology we are using. For example, individual consumers using RabbitMQ can observe out-of-order messages when multiple subscribers requeue messages.[14] Network partitions can also cause ordering issues depending on the broker's configurations.

Resilience strategies can also impact ordering. For example, depending on the way retries are implemented, the order can be lost when retrying a message. We often discussed the ability to replay an event stream to rebuild a view of the data. In a way, expected or not, it jeopardizes message ordering since we will start to consume older messages.

Concurrency can, and often will, play a factor in ordering events. Even if events are published and stored in order in the event stream, two instances of the same service (or even two threads of the same service) concurrently consuming the messages can unorder them due to different processing rates. Figure 6-6 illustrates this example.

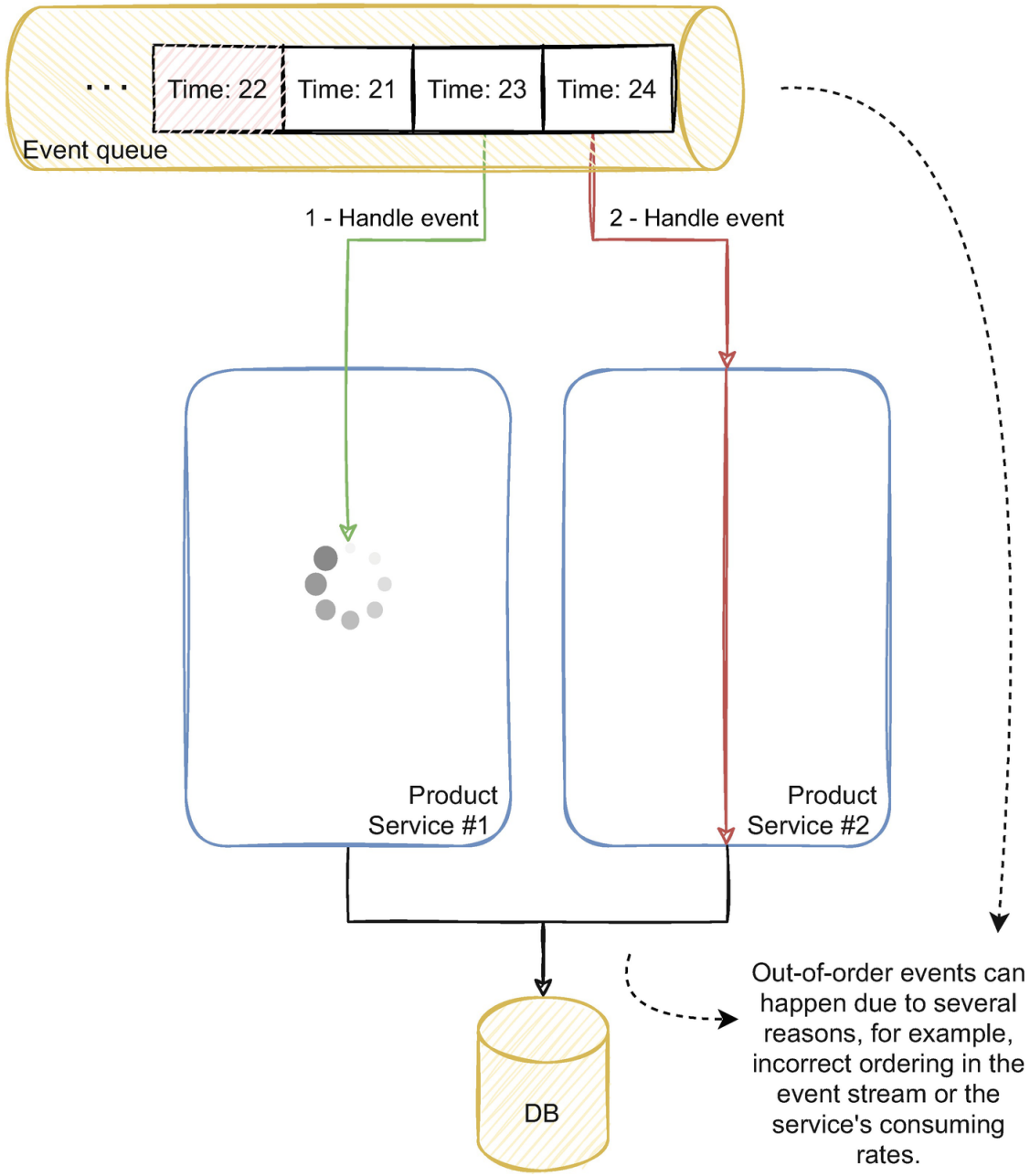

Figure 6-6 Two instances of the same service consuming from a queue with out-of-order events and different consumption rates

Two events might have different operations depending on the event's processing logic. Even if the two events have the exact same operations, the associated data will likely be different, which implies different times in fetching and handling that data. Two instances, or even two threads, will execute the same operations at slightly different rates (due to the associated data and even due to the underlying hardware and operating

system characteristics). Consuming both events concurrently is enough for one consumer to occasionally handle a concurrent older event faster than a more recent one.

6.5.2 Solving Out-of-Order Events with Versioning

Since events can lose their order due to several factors, what can we do to prevent invalid states? One way is using event versioning. Events reflect a change that happened to an entity. Conceptually, the event's version is the version of the entity at the time of the change. For example, if our entity is a product, a product created event could have version 1; if someone changed the product's category, it would publish an event with version 2; and so on.

The concept of a sequential version per entity relates to the DDD aggregate concept. The entities are related to the aggregates we define in the domain. Every time an entity changes, the version increases. Each version represents the state of the entity at that given moment in time. Events can use the same version and relate to the state change that triggered the event.

However, managing a sequential, unique, and incremental version can be tricky[15] in distributed systems. Twitter Snowflake,[16] for example, highlights the challenge of generating unique ids in a distributed database like Cassandra. Consistently generating these ids can also have performance impacts in heavy workloads. A timestamp, or using the number of milliseconds since the epoch, for example, can be a valid alternative. This way, each service can easily generate a version always higher than the previous, although not sequential.

A sequential version can be relevant in partial events where the event doesn't have the entity's full information. For example, if the inventory service published stock in events and stock out events, a service building the product's stock couldn't skip any version. Otherwise, the stock wouldn't add up to the correct value. In those use cases, services often need to consume every event in the stream in the exact order (can't skip

versions). When a service exhibits this behavior, it can be harder to reason with and is often less resilient, as we will discuss in Chapter 7.

The way we design events can also have an impact on the importance of order. We would need to consume every stock in and stock out in the stream to have the latest stock quantity (this has other implications; for example, we would need to allow negative stock at least temporarily). But notice that a stock in and stock out event, contrary to a stock change, aren't idempotent, which is an important property to have, as we will discuss in Chapter 7.

Event versioning provides a way to understand if the event the service is processing is older or more recent than our internal state. In the example we discussed with the inventory service in Figure 6-6, when consuming the event with version 21, we could detect that the current version of the product's stock is 22. Then we could decide on what to do.

Some business processes might benefit from still processing the event or all events that arrive late within a given window. For example, if we needed to send a notification every time a product has one last stock unit, the ordering wouldn't be that relevant and we might want to process every event despite the order the events arrive. However, in this example where we are updating the current stock for each product, we are only interested in the latest state, so we could ignore older events.

Notice that even with event versioning and validating the version on each event, we wouldn't absolutely guarantee that we wouldn't process older versions. Two services could be processing sequential versions concurrently and process the most recent version faster than the older one. We would still have to combine this with one concurrency prevention strategy we discussed before.

Versioning events is always relevant, and, either by using an aggregate version or a timestamp, it is useful for consumers to use. It is important to manage idempotency and out-of-order events on the consumer side. We can also delegate the responsibility to order events to the event broker by routing messages to specific partitions, as we will discuss in Section 6.6.

6.6 Using End-to-End Message Partitioning to Handle Concurrency and Guarantee Message Ordering

Until now, we discussed solutions to ordering and concurrency using by implementation strategies. We discussed patterns we can apply to solve the concurrency and ordering issues. A better and more sustainable approach is to avoid concurrency and ordering issues altogether, solving concurrency by design. This section will work through a similar example to the ones we discussed in this chapter and detail the inner workings of a message stream and how we can partition messages to avoid concurrency altogether.

Let's work through the same example we discussed before with the inventory service handling multiple order created events. Instead of just focusing only on the inventory service, let's look at the flow since the order service. Figure 6-7 illustrates this flow.

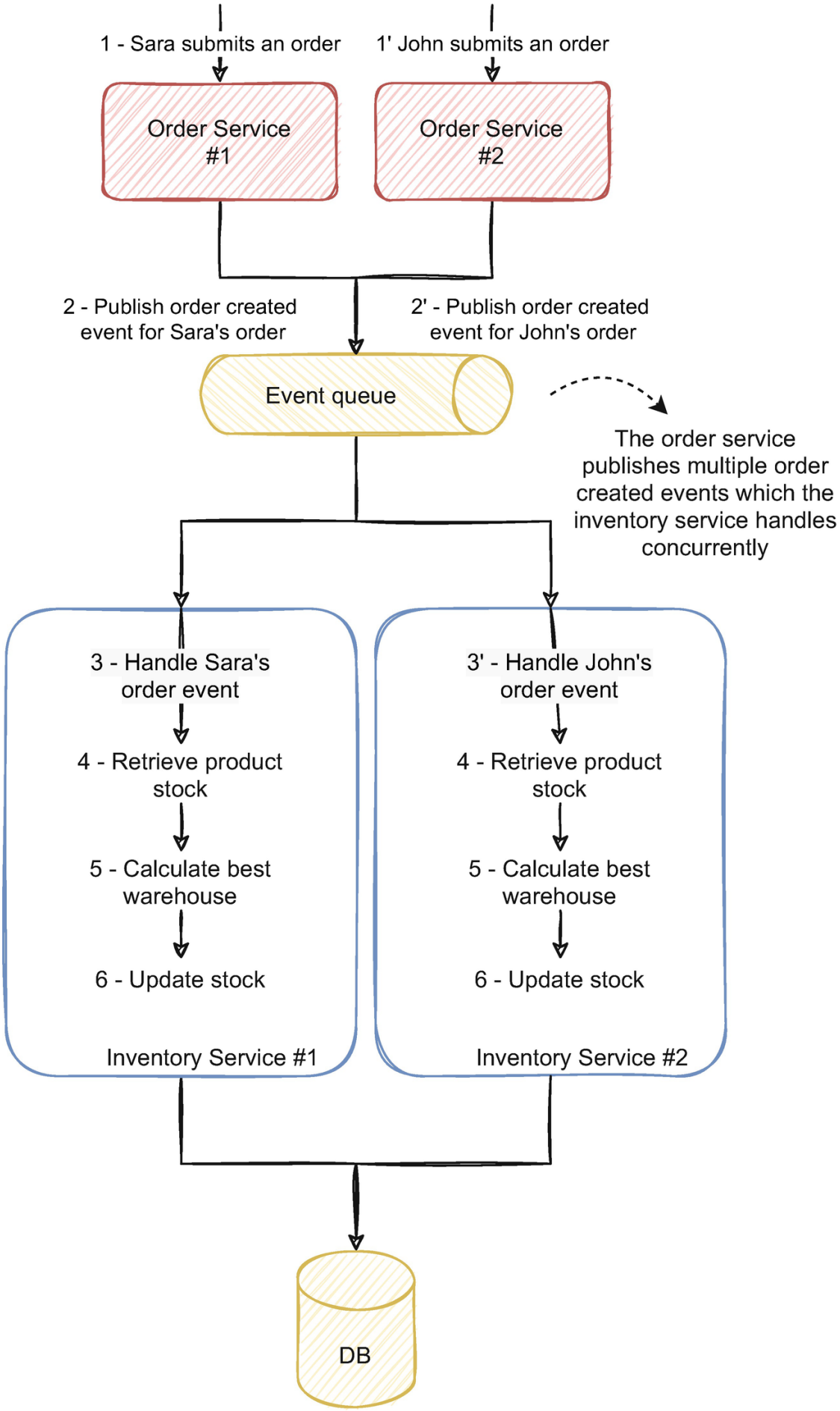

Figure 6-7 Multiple instances of the order service process multiple orders. The inventory service handles concurrently the events for those orders

The order service, such as the inventory service, can be scaled horizontally and can have multiple instances; in Figure 6-7, each of them has two. Each instance of the order service handles multiple order requests; in this case, Sara and John both submit an order. The order service handles the requests and publishes events signaling the order was created and is ready to follow the order fulfillment flow. The inventory service handles those events and changes the stock according to those orders. Since both John and Sara ordered the same product, which only has one stock unit left, processing both orders simultaneously can produce a concurrency issue the same way we discussed before.

How can we solve this by partitioning messages? To understand this concept, it's important to understand how the message broker works internally. To illustrate this, let's look at Kafka and detail how it arranges messages.

6.6.1 Real-World Example of Event-Driven Message Routing Using Kafka

In the example in Figure 6-7, the order service produces events to an event queue; in Kafka, it's called a topic. The inventory service subscribes to that topic and consumes the events. Events are stored durably and remain available after consumption (as we discussed in Chapter 3). The order service has multiple producers (since there are several instances of the order service) and has multiple consumers, in this case, the various inventory service instances.

In this case, the only consumer is the inventory service, but there could be other services also with multiple instances reading from the topic. The anatomy of a Kafka topic and its interactions with different services is illustrated in Figure 6-8.

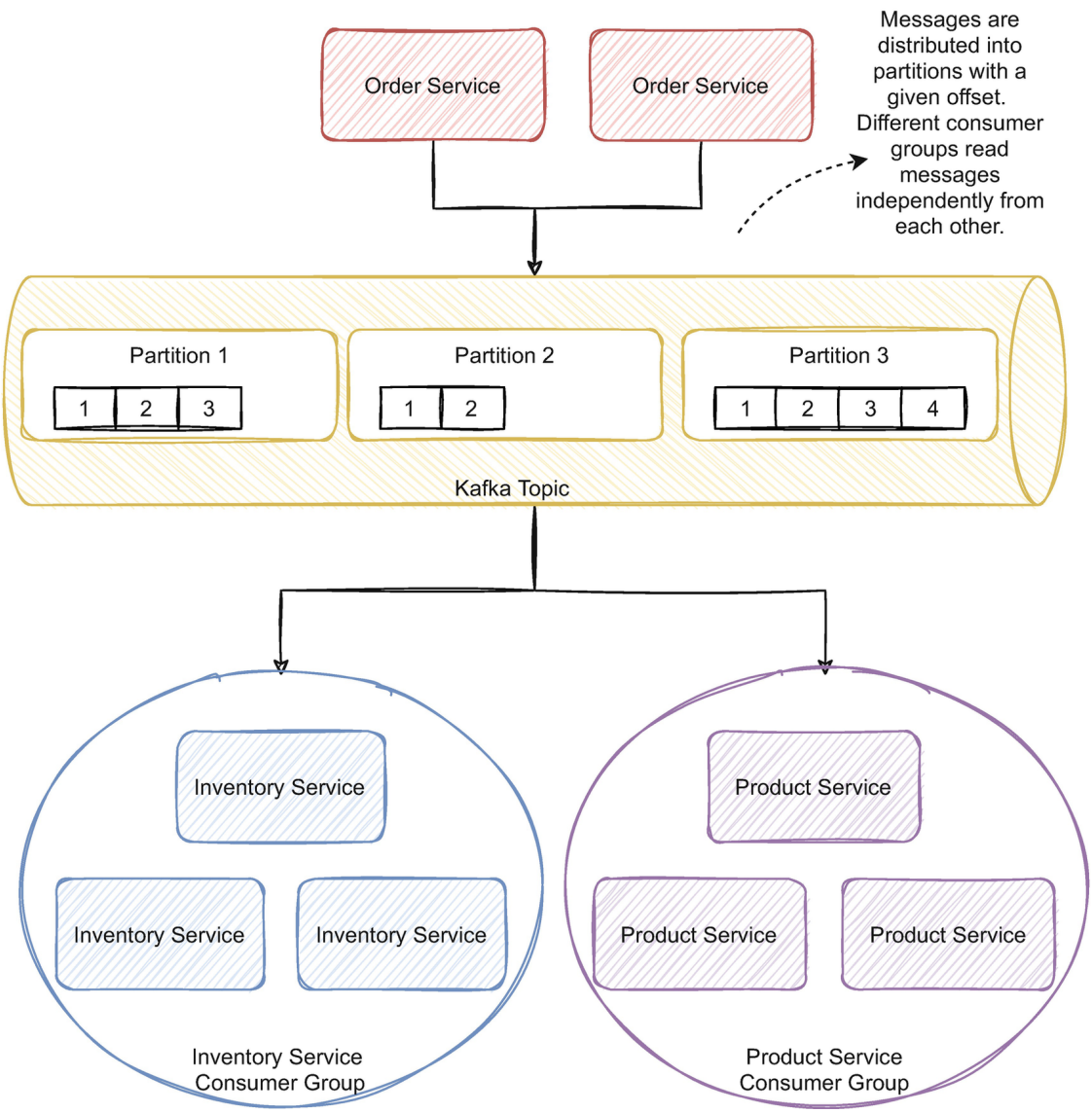

Figure 6-8 Detail of a Kafka topic and its interactions with other services

The order service publishes messages to the Kafka topic. Kafka distributes those messages to different partitions. Partitions enable good scalability since they allow producers to write data, and consumers to read, to multiple nodes at the same time. Partitions are also crucial for high availability; partitions can be replicated across several machines to guarantee fault tolerance.

Topics are basically a distributed append log; when services publish a message, the broker appends it to the end of the stream. Each message has an offset which represents the message position in the stream for each partition. For example, if a given partition has ten messages, when a producer writes a new one at the end of the stream, it will have offset 11.

Consumers read from the topic and consume messages from the different partitions. Messages are distributed to service instances with the same consumer group, and each

consumer group acts as a single consumer. The example depicted in Figure 6-8 shows three instances of the inventory service and three instances of the product service. Instances of the same services have the same consumer group; different services have different consumer groups. For example, each of the three instances of the inventory service consumes different messages; they advance in the stream as a group; thus, different instances with the same consumer group never receive the same messages. Different consumer groups will consume from the stream independently and concurrently. For example, the message with offset 1 will be delivered to inventory service's instance 1 and product service instance 2, but never to more than one service instance within the same consumer group. Figure 6-9 illustrates the offsets of the two services.

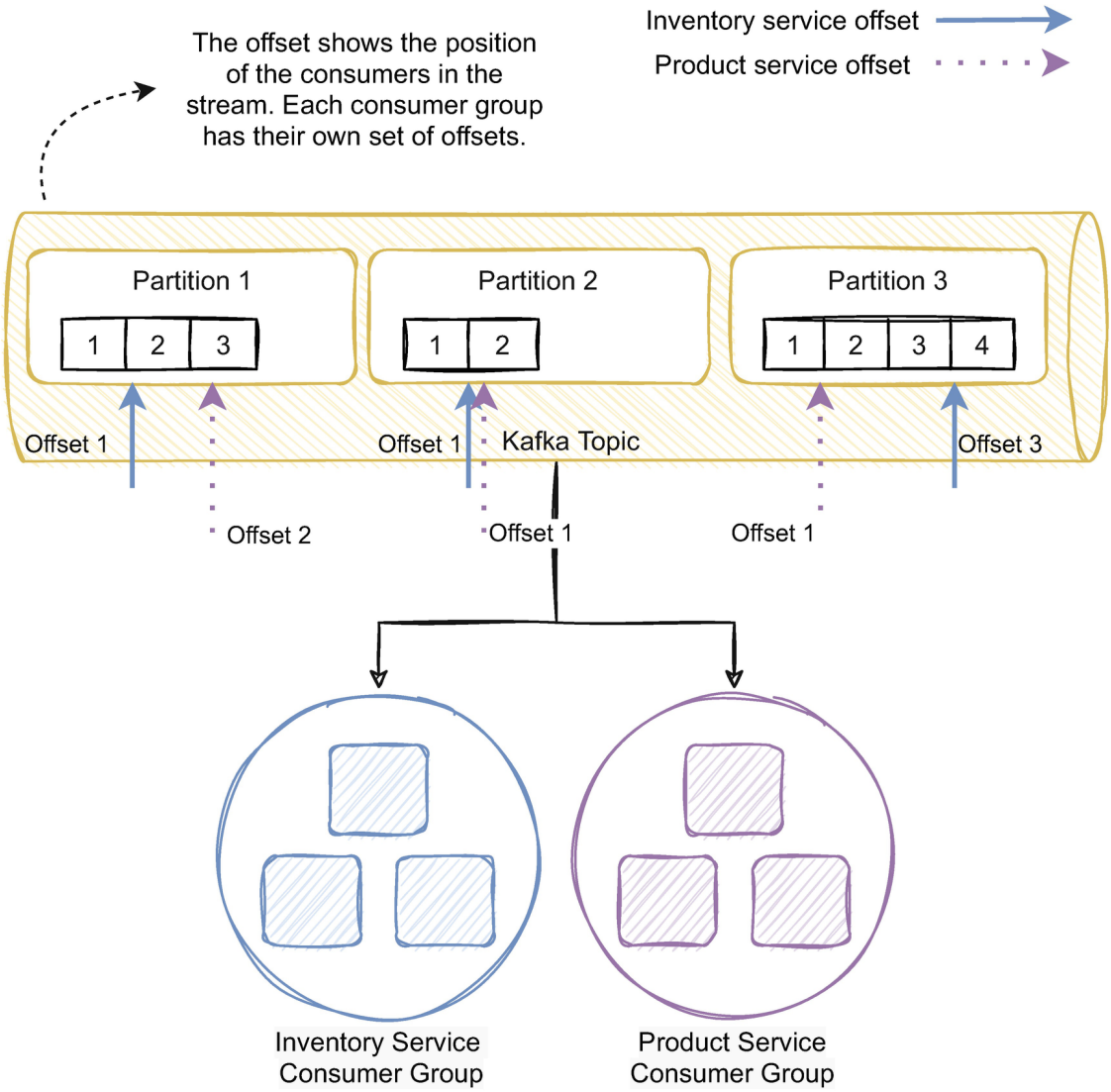

Figure 6-9 Offset detail of each consumer group for both services

The arrows in Figure 6-9 illustrate the consumer's position in the stream. In fact, in Kafka, consumers are cheap; you can reason as if they were a pointer in the event stream – opposed to ephemeral message brokers, where consumers often require an additional effort by the broker

and often struggle if consumers start to pile up messages in the queues. As illustrated in Figure 6-9, consumers only advance in the stream, and messages remain in the topic and aren't removed. If a new service needed to consume from the stream, it would have the messages available. If the inventory service needs to rebuild its state, it could merely set its offset to zero and reread the stream.

The topic's number of partitions is an essential factor to the scalability and performance of the system. Kafka assigns a number of partitions to each consumer. For example, if a topic has six partitions and the inventory service has three instances, it will likely be assigned two partitions to each inventory service instance. In the example in Figure 6-9, we have three partitions to three service instances; each instance will likely have one partition. However, what if we decided to further horizontally scale the inventory service by adding one more instance? Although one consumer can consume messages from several partitions, each partition is consumed by exactly one consumer. So if we add another instance to the inventory service, there would be no partition to assign it to; hence, the new instance wouldn't have any load whatsoever. We wouldn't be able to scale the service further.

We can change the number of partitions after the topic is created, but adding partitions doesn't change the partitioning of the existing data, which might be troublesome to the consumers as we will see in the next section. The number of partitions limits our ability to scale; an arguably reasonable decision when creating a topic is to plan beforehand the scalability needs of the topic and create it with a larger number of partitions than it currently needs. This planning will give space to scale the service in the future.

Kafka has two pivotal characteristics that can help to deal with out-of-order events and concurrency. In a partition, Kafka guarantees events are read in the same order they were written. This guarantees ordering on the broker side; events can still lose the order on the consumer side, as we discussed in Section 6.5. It is essential to notice that Kafka only guarantees ordering inside a partition. A topic has several partitions, so ordering is not guaranteed across partitions and throughout the whole topic.

However, when publishing, we are also able to define the event's routing key. The broker ensures it delivers events with the same routing key to the same partition. This property is a fundamental characteristic we can use to avoid concurrency and out-of-order messages as we will detail in the next section.

6.6.2 The Relevance of Message Routing and Partitioning in Event-Driven Microservices

At the beginning of the section, in the example in Figure 6-7, we discussed how the inventory service could have a concurrency issue by simultaneously processing the order service's events. The concurrency issue occurs because two instances of the inventory service process order for the same product at the same time; in this case, both John and Sara ordered the same product. John's order was processed by one instance and Sara's order by another instance.

But how would this example play out if we defined a routing key in the order service? Every message with the same key will be routed to the same partition by specifying a routing key. As we discussed, each partition has exactly one consumer. If we use the product id as the routing key, every order for the same product would be routed to the same partition and consequently to the same consumer instance. Figure 6-10 illustrates this flow.

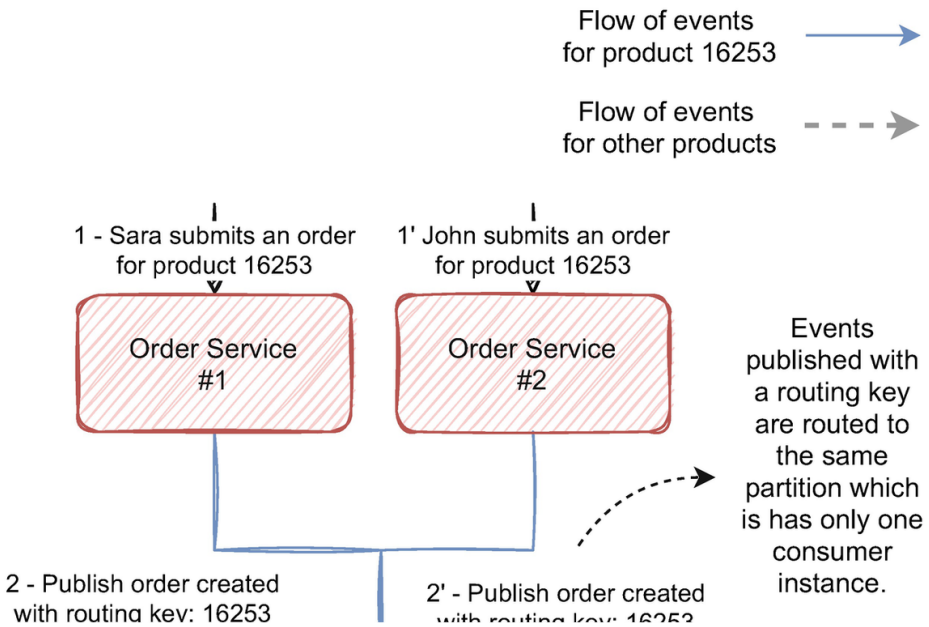

Figure 6-10 Flow of the order events when published with a routing key

John and Sara are both trying to buy product 16253, which only has one stock unit left. The order service publishes events using the product id as the routing key, in this case, 16253. Since both events have the same partition key, they are both routed to the same partition, in this case, par-

tition 1. Each partition has only one consumer; inventory service instance 1 is handling events from partition 1, so this instance will assuredly receive both events. The instance will also receive the events by the order they were published since ordering is guaranteed in the same partition.

Looking just at this example, we might think the other service's instances won't receive any load, but they will continue to receive events, just not for that product id. The other instances are still processing other products with other ids. This strategy guarantees every event with the same partition key will be processed by the same instance, but every instance will continue to process events for the partitions assigned to them; hence, they will continue to process other products. We can reason as if we assigned a range of products to each instance; events from products inside the range will be processed only by the corresponding instance.

Using this strategy, we just transformed the distributed concurrency issue into a local one. Is this sufficient to eliminate concurrency and ordering issues altogether? Each service instance will likely have parallelism, so concurrency issues can still occur inside each instance. For example, we might receive John's and Sara's order in the correct order and in one instance, but two threads might process the events concurrently inside the instance. However, this way, there isn't the need to synchronize the several instances through the database or a distributed lock manager. By routing messages, we avoid distributed concurrency issues and only deal with local ones, where the traditional mechanisms to deal with concurrency are relevant.

6.6.3 Using End-to-End Partitioning to Handle Concurrency and Ordering

Distributed synchronization is complex and is often the root of complicated issues and corner cases. It isn't arguably suited to the distributed event-driven mindset since it creates strong dependencies between the instances. Partitioning messages is one way to avoid distributed synchronization through architecture design. However, as we discussed, even in a local scope, concurrency is still an issue. Solving concurrency by implementation often implies complicated developments and is often hard to

reason with and to maintain. This subsection will take one step further the partitioning approach and detail how we can avoid concurrency altogether.

To handle concurrency in the service's local scope, we can use the traditional approaches like an in-memory lock or one of the techniques we discussed in Sections 6.3 and 6.4. However, an arguably good way to handle concurrency by design is to follow the same approach we discussed when we detailed the broker's event routing. Following this approach, we can route the events we receive in the service to specific paths. Figure 6-11 illustrates this situation.

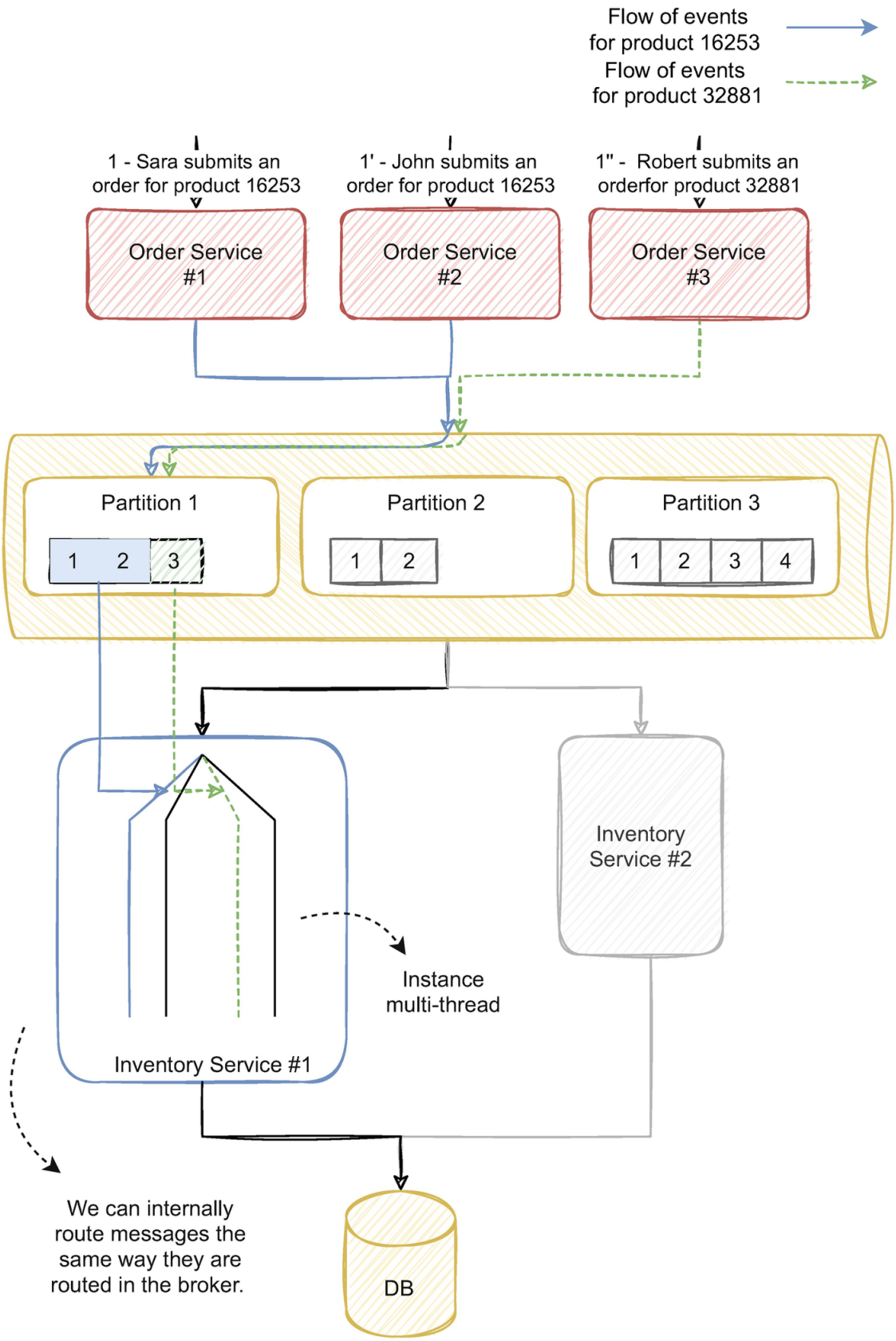

Figure 6-11 Flow of the events from the producer to the consumer using end-to-end partitioning

Inside instance 1 of inventory service, we detailed several thread paths that correspond to the service's parallelism. In this case, the service processes four events in parallel. Figure 6-11 highlights two paths, the path for the two events of orders for the same product (with id 16253) with the solid line and the path for a third order for the product with id

32881 with a dashed line. The three instances of the order service publish the three events using the product id as the partition key. The broker routes the events to partition 1, which is assigned to the first inventory service instance. The inventory service can handle the three events concurrently since the parallelism degree can process four events simultaneously.

However, when handling the events, we can route them to specific thread paths based on a routing key. We can forward the events with the same key to the same thread so we can process them sequentially. By handling them in a single-threaded manner, there are no concurrency issues. The service itself is multi-thread since the other threads will process different ranges of routing keys. For example, events with product id 32881 will be processed simultaneously with events with product id 16253 (as illustrated in the diagram), but the two events for id 16253 will be processed sequentially; the second will only start to be consumed when the first finishes. We can route the messages using the same routing key the broker uses or even the partition number, as long as the number of partitions is higher than the number of parallelism of all instances.

If we guarantee the order service publishes events in order, we also avoid ordering issues. The broker ensures that inside a partition, the order is maintained; if we have the same consideration when assigning messages in-memory threads, we maintain the end-to-end ordering.

With end-to-end partitioning, we can achieve greater performance by enabling parallelism inside and outside of the service and tune the parallelism to take the most out of the physical resources while completely avoiding concurrency and ordering issues. It also avoids the dependency of an external dependency, locks of any sort and retries. This approach has high synergy with the event-driven mindset; reasoning with the service is always in the context of the event flow. It is an exceptionally simple way to use an event-driven mindset to completely remove ordering and concurrency concerns without performance overhead. Developments can be implemented in a no concurrency context which greatly reduces the complexity and allows developers to focus on the business logic rather than technical concerns.

6.6.4 Limitations of End-to-End Partitioning in Event-Driven Microservices

As we discussed, end-to-end partitioning entirely avoids concurrency and guarantees ordering. In a fully event-driven architecture, it can be an excellent approach to apply consistently across services, but it does have some limitations in some use cases. It also has some inconspicuous caveats that are easy to miss. This subsection will discuss those limitations and propose possible solutions.

Hotspotting

The event broker uses the partition key to route events to one partition; hence, every event with the same key will be routed to the same partition. You might imagine what happens if a large percentage of the entire set of events has the same key; the data distribution will be largely unbalanced.

For example, in the use case we discussed until now, we used the product id as the partition key. But imagine we needed to guarantee concurrency across product categories, and we didn't want to process products simultaneously with the same category. We could use the category id as the partition key. Let's say most of our product catalog is composed of clothing, while the other categories, like shoes and accessories, represent a considerable small fraction of the entire catalog. By this design, most of our events would have the clothing category id. Since every event with the same id would be routed to the same partition, the partition with the clothing products would be fairly larger than the remaining partitions.

This unbalance between the partitions is called hotspotting. As you might recall from the previous sections, the number of partitions is pivotal to performance and scalability concerns. In this case, a large percentage of our data would be assigned to a single partition which only has one consumer by the broker's design. This unbalance also means a single instance would handle a large portion of our events, which will undoubtedly degrade the system's performance. It also limits the ability to scale since no more consumers can be assigned to that partition. When the number of messages is extensively large, the broker might also struggle with it; Kafka, for example, has to fit one partition in a single machine. If

we have a considerably large number of messages, it might be impracticable or costly to fit them in one machine, and the broker might struggle to deal with it properly.

In our example, we used the product id as a routing key; it is natural that some products are more sought-after than others and thus have more orders. Having more orders also means more events with the same key. However, with a large number of products, the number of messages in the partitions will eventually even out; products with less orders will compensate for the ones that have more.

Two important considerations when choosing a partition key are the key's granularity and diversity. The partition key needs to be granular enough to accommodate for an even distribution across partitions. It also needs to have enough diversity (several times greater than the number of partitions) and sufficient different values to enable adequate routing options.

There might be use cases where an adequate routing key is not available (as we discussed with the category's example), but this is often the exception. Event-driven systems often imply a large scale, which, despite all its challenges, provides adequate data diversity to enable suitable routing keys. However, be mindful when choosing a routing key.

Momentary Hotspots

We often think of solutions in their clean and platonic state. If we receive three events and have three partitions, the broker will route one event to each partition. Balanced, as all things should be. However, in the real world, things are messy, unpredictable, and often chaotic. Solutions might not behave as we expect they would all of the time.

Although most of the time predictable, traffic can be erratic and unpredictable patterns might occur from time to time. For example, in the example we discussed in this section, we published order events and used the product id as the partition key. Most of the time, traffic might be somewhat stable. However, imagine we drop a highly sought-after product in our platform, which users order intensively. This will temporarily

generate a large amount of events with the same routing key, creating a momentary hotspot.

The performance of the system might drop while this is occurring if the load is sufficiently disparate. This is often a rare event and one that the system might afford. However, we should understand its implications and understand if there are relevant consequences for the business. We should also design our systems not to produce momentary hotspots often. For example , we should avoid recovery processes that unload an impactful number of events with the same key.

The Mix of Event-Driven and Synchronous APIs

An apparent limitation of the end-to-end partitioning approach is when the service needs to handle events and process synchronous requests through an API. Although this kind of services might be the exception in an event-driven architecture, they are relatively common and might make the end-to-end partitioning approach inviable. This use case is illustrated in Figure 6-12. We should avoid these kinds of configurations and preferentially use a fully event-driven approach or a synchronous API; having both can be particularly challenging, especially when we need to prevent concurrency and ordering issues. Having said that, in the real world, we might find this kind of approach. In this topology, we can't directly apply end-to-end partitioning since routing and mixing synchronous requests with events might be unsuitable.

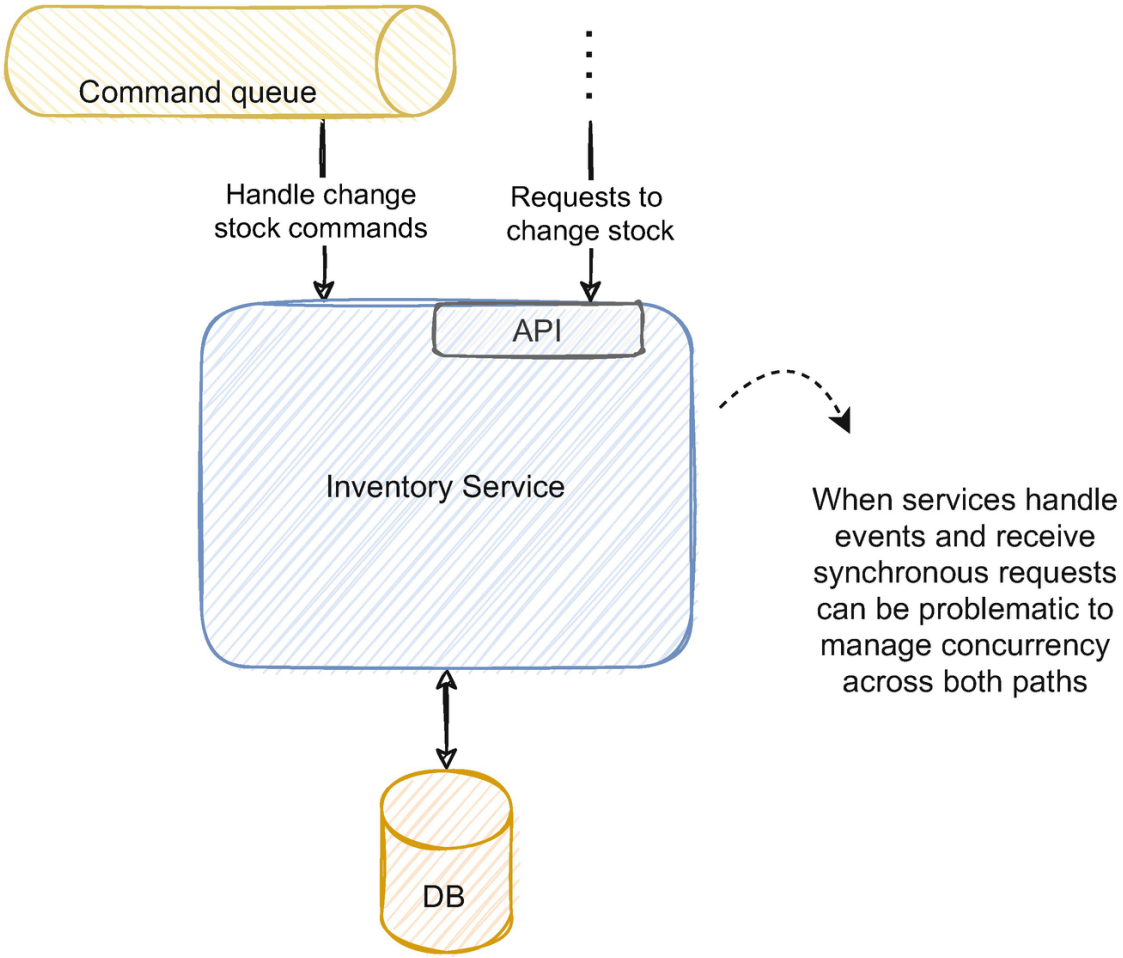

Figure 6-12 Service handling events and processing synchronous requests

One possible approach is to convert the synchronous request into a command and publish it to the queue. For example, the inventory service would receive a POST request to change stock, but instead of synchronously satisfying the request, it would send a command to the command queue. This way, we would have only one entry point, and it would be possible to route the commands accordingly. However, this approach might not be viable if the API needs to respond synchronously. If this is the case, we can still apply a pessimistic or optimistic concurrency prevention strategy to manage both scenarios.

6.7 Summary

- Handling concurrency in a single application has different implications than handling it in a distributed architecture. The traditional strategies to prevent concurrency are often unsuitable for distributed event-driven services.

- We can solve concurrency by implementation or by design. Implementation strategies often involve either optimistic or pessimistic approaches. The end-to-end partitioning pattern can be an effective way to handle concurrency by design in event-driven systems.
- Optimistic concurrency prevention strategies shine in environments with low chances of concurrency. They don't involve locking and often require retrying the process when concurrency is detected.
- Pessimistic concurrency prevention strategies are suited to environments with high chances of concurrency; they lock a given resource and block access. They often rely on an external dependency to manage the synchronization.
- We can achieve pessimistic concurrency with distributed locks by resorting to an external tool like Consul or ZooKeeper. Instead, we can also use some database technologies or use transactions.
- Out-of-order events can be troublesome and jeopardize the consistency of the system. We can use event versioning to tackle ordering issues.
- End-to-end partition is a way to solve both concurrency and ordering issues by architecture design. In event-driven architectures, it's arguably our go-to solution to solve these issues. It can be a clean, performant strategy to handle ordering and concurrency.
- Hotspotting can be an impactful consequence of routing events. We should be mindful of this issue when defining the partition key and design the routing strategy accordingly.

Footnotes

1 Full article exploring this concept in Maurice Herlihy, "Apologizing Versus Asking Permission: Optimistic Concurrency Control for Abstract Data Types," March 1990, www.researchgate.net/publication/234778080_Apologizing_Versus_Asking_Permission_Optimistic_Concurrency_Control_for_Abstract_Data_Types

2 Full technical reference in Elastic.co, "Optimistic concurrency control," www.elastic.co/guide/en/elasticsearch/reference/current/optimistic-concurrency-control.html

3 Full technical reference in Datastax.com, “Using lightweight transactions,” https://docs.datastax.com/en/cql-oss/3.3/cql/cql_using/useInsertLWT.html

4 More details in Sandeep Yarabarla, “Learning Apache Cassandra - Second Edition,” April 2017, https://learning.oreilly.com/library/view/learning-apache-cassandra/9781787127296/5e5991cb-eb1e-4459-9114-1d86e974e927.xhtml

5 GitHub project and details here: https://github.com/NEventStore/NEventStore

6 Full technical reference in “Handling Concurrency Conflicts (EF6),” October 23, https://docs.microsoft.com/en-us/ef/ef6/saving/concurrency

7 Referenced in ZooKeeper documentation in “ZooKeeper Recipes and Solutions,” https://zookeeper.apache.org/doc/r3.5.5/recipes.html#sc_recipes_Locks

8 Further details in Redis.io, “Distributed locks with Redis,” https://redis.io/topics/distlock

9 Full article in Martin Kleppmann, “How to do distributed locking,” February 8, 2016, https://martin.kleppmann.com/2016/02/08/how-to-do-distributed-locking.html

10 Further details in aphyr.com, “Jepsen: etcd and Consul,” June 9, 2014, https://aphyr.com/posts/316-jepsen-etcd-and-consul

11 Full analysis in consul.io, www.consul.io/docs/architecture/jepsen

12 Full talk in Kyle Kingsbury, “GOTO 2018 Jepsen 9: A Fsyncing Feeling,” May 8, 2018, www.youtube.com/watch?v=tRc0O9VgzB0&t=1526s

13 Further details in Microsoft documentation, March 14, 2017, https://docs.microsoft.com/en-us/sql/relational-databases/system-stored-procedures/sp-getapplock-transact-sql?view=sql-server-ver15

14 Further details in RabbitMQ documentation, “Broker Semantics,” www.rabbitmq.com/semantics.html

15 Further details in João Reis, “Unique Integer Generation in Distributed Systems,” September 20, 2018, www.farfetchtechblog.com/en/blog/post/unique-integer-generation-in-distributed-systems/

16 Further details in Twitter blog, Ryan King, “Announcing Snowflake,” June 1, 2010, https://blog.twitter.com/engineering/en_us/a/2010/announcing-snowflake.html

---